



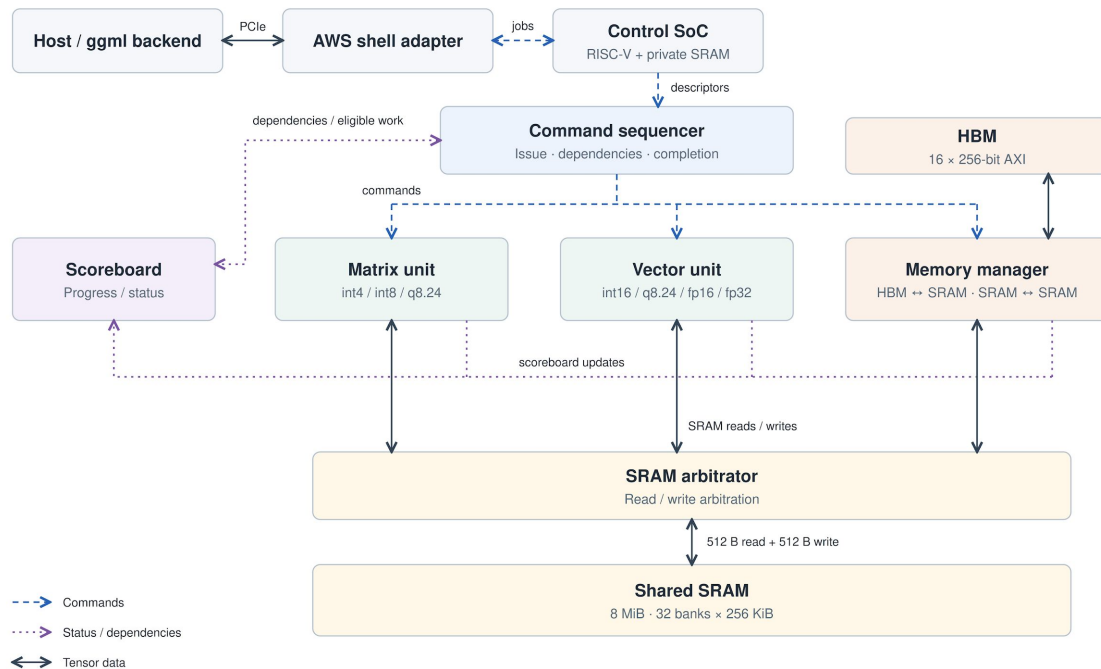
Compute Team Meeting

Matrix and vector unit microarchitecture
Silicon Badgers



BPU block diagram

BPU Block Diagram



Data types and scaling

Operand	Format	Use / fraction bits
Weights	Signed INT4 or INT8	Dense projections
Activations	Signed INT8	Dense projection inputs
Raw dot sums	INT32	Dense accumulation
State S , v , Δ	INT32 Q8.24	Recurrence; value = code $\times 2^{-24}$
Q and K	Signed INT16.Ff	f = 14–24
Decay deficit	UINT16.Ff	f = 15–40
Beta	UINT16.F15	15 fractional bits

Block scaling will be included; its implementation and parameters are undecided

Fixed-point notation and shared scales

Notation	Stored code	Value	Scale in the repo
INT16.Ff	Signed 16-bit	$\text{code} \times 2^{-f}$	Q/K: shared $f = 14-24$ per token and head
UINT16.Ff	Unsigned 16-bit	$\text{code} \times 2^{-f}$	Decay: $f = 15-40$ per token and head
Q8.24	Signed 32-bit	$\text{code} \times 2^{-24}$	State and values: fixed $f = 24$

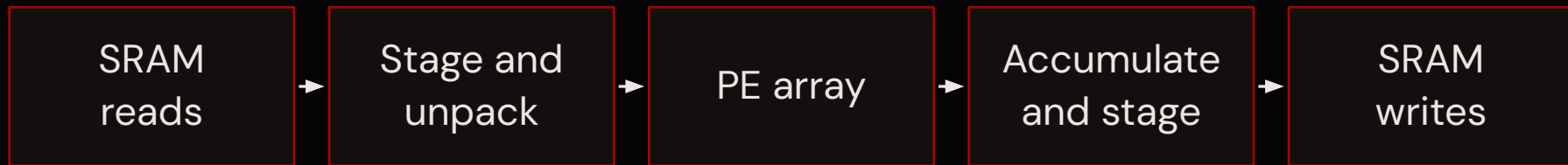
Ff means scale 2^{-f} ; Q/K share a dynamic f across 128 codes (block floating point)

$$\text{Q8.24} \times \text{INT16.Ff} \rightarrow \text{RNE}(\text{product code} / 2^f) \rightarrow \text{Q8.24}$$

RNE rounds to nearest; halfway cases round to the even integer

Proposed matrix unit datapath

Commands specify the operation, operand views, dimensions, dtypes, fraction bits, and arithmetic profile



Dense operations use narrow integer products

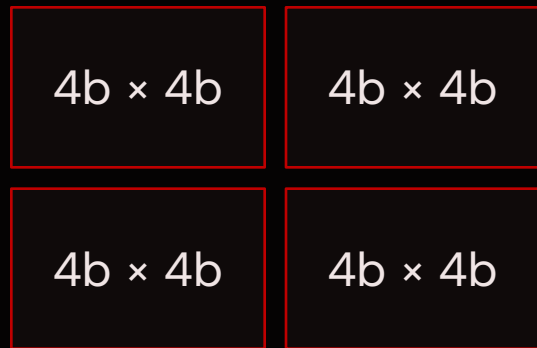
Recurrence uses serial wide fixed-point products

The matrix unit reports completion and range errors to the scoreboard

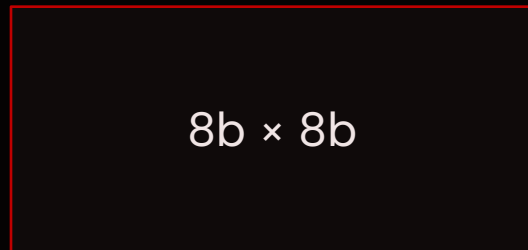
The array size, buffering, and dataflow are undecided

PE multiplication modes

2×2 INT4 mode



INT8 mode



Mode control configures the same PE arithmetic.

The proposed Q8.24 mode retains operands and combines narrow partial products over multiple serial steps.



Example matrix ISA: dense operations

These examples use the repo's proposed command names and the BYTE_DOT profile

Use	Command	Operands	Example
Prefill	M.GEMM	$\text{INT8} \times \text{INT4}$ or $\text{INT8} \times \text{INT8}$	$C \leftarrow \text{M.GEMM}(A, B)$
Decode	M.GEMV	$\text{INT8} \times \text{INT4}$ or $\text{INT8} \times \text{INT8}$	$y \leftarrow \text{M.GEMV}(W, x)$
Rank-one product	M. OUTER	$\text{INT8} \times \text{INT4}$ or $\text{INT8} \times \text{INT8}$	$C \leftarrow \text{M.OUTER}(a, b)$

BYTE_DOT produces exact INT32 sums



Block scaling will be included, but its implementation remains undecided

Serial multiplication profiles

Profile	Operand pair	Product bits	RNE shift
QSTATE_S16	$Q8.24 \times \text{signed INT16.Ff}$	48	f
QSTATE_U16	$Q8.24 \times \text{UINT16.Ff}$	48	f
Q824_ORDERED	$Q8.24 \times Q8.24$	64	24

The proposed PE combines partial products serially with sign correction and no intermediate rounding

Complete product $\rightarrow$ RNE shift $\rightarrow$ INT32 saturation $\rightarrow$ ordered accumulation

The serial multiplier schedule and cycle counts are undecided

Example matrix ISA: recurrence

The matrix commands run after the decay step and use the QSTATE_S16 profile

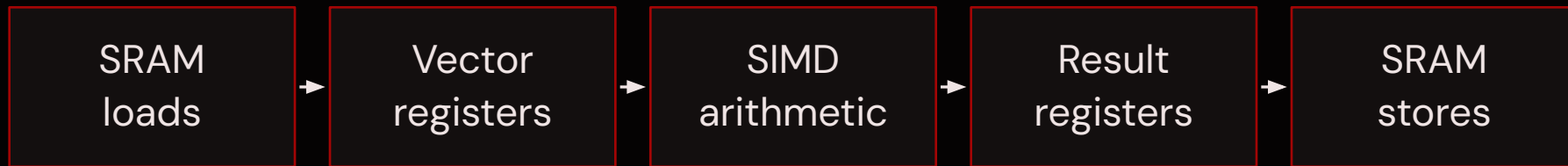
Step	Command	Example
Read memory	M.GEMV	$\text{mem} \leftarrow \text{M.GEMV}(S^T, k, \text{shift}=kf)$
After error and Δ Update state	M. OUTER	$S \leftarrow \text{M.OUTER}(k, \Delta, \text{shift}=kf, \text{accumulate}=\text{true})$
Read updated state	M.GEMV	$\text{out} \leftarrow \text{M.GEMV}(S^T, q, \text{shift}=qf)$

Each product is rounded separately, and reductions follow increasing logical K

State is committed after every token, with dependent tokens kept in order

Proposed vector unit datapath

A local RISC-V core controls SIMD operations and SRAM transfers



The datapath supports FP16/FP32 and integer operations

Reductions and nonlinear functions can use shared resources

The command sequencer dispatches vector work and tracks completion

Undecided: lane count, register capacity, and shared-resource sizing

SIMD execution

- A tiny RISC-V core controls the SIMD lanes with custom instructions
- Loads must complete before their registers are consumed
- Inactive stores write nothing, and tails stay within the operand view
- The sequencer checks scoreboard status before issuing dependent work

RISC-V control of SIMD lanes

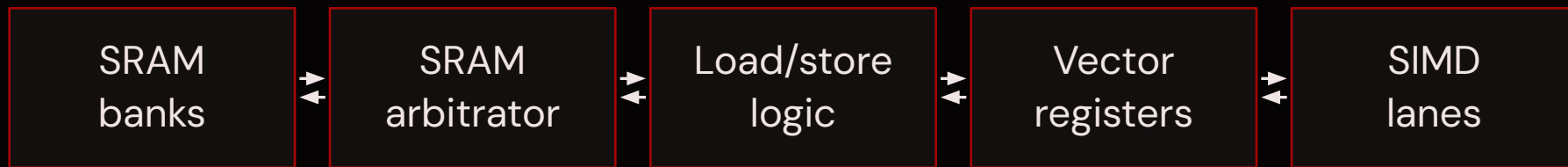
A local RISC-V core runs scalar code and issues custom SIMD instructions

Control	Instructions	Inputs	Effect
Active lanes	U.SETVL	Vector length and predicates	Select lanes for the current chunk
Memory	U.LOAD / U.STORE	SRAM view, index, register	Move typed data between SRAM and registers
Iteration	BEQ / BNE / BLT	Scalar registers	Use ordinary RISC-V branches for loops
Completion	Interface undecided	Local status	Report completion after SRAM stores finish

The sequencer dispatches both units; they exchange data through SRAM

SRAM loads and stores

U.LOAD and U.STORE move typed vectors between SRAM views and registers



A load fills registers before arithmetic uses them

A store writes results from registers to SRAM

Load/store logic computes addresses from the base, strides, and chunk index

Undecided: request widths, buffering, bank mapping, and timing

Example vector ISA: arithmetic

These are proposed commands; binary encodings and timing are unspecified

Use	Commands	Types	Behavior
Floating point	V.ADD/SUB/MUL V.FMA	FP16 / FP32	Elementwise arithmetic
Integer	V.ADD.SAT/SUB.SAT V.AND/SHL/SAR	INT32 / UINT32	Saturation and bit operations
Reduction	V.REDUCE	FP16 → FP32 INT32 → INT64	Sum, max, absmax, or argmax
Conversion	V.CAST V.PACK/UNPACK	Float / fixed point INT4 / INT8 / INT16	Convert or pack operands

Profiles define rounding, saturation, and reduction order; FMA rounds once

Example vector ISA: recurrence

S and v use Q8.24; d is a UINT16 decay deficit, and b is UINT16.F15

Step	Command	Example
Decay	V.MULSHIFT.SAT V.SUB.SAT	$\text{decay} \leftarrow \text{V.MULSHIFT.SAT}(S, d, df)$ $S \leftarrow \text{V.SUB.SAT}(S, \text{decay})$
Error	V.SUB.SAT	$\text{error} \leftarrow \text{V.SUB.SAT}(v, \text{mem})$
Update	V.MULSHIFT.SAT	$\text{delta} \leftarrow \text{V.MULSHIFT.SAT}(\text{error}, b, 15)$

V.MULSHIFT.SAT uses an exact 48-bit product, an RNE shift, and INT32 saturation

The sequencer orders recurrence stages, with results passed through SRAM



Composed vector operations

RISC-V routines combine SIMD primitives under a declared arithmetic profile

Use	Commands	Profile	Purpose
Nonlinear	V.EXP/SILU V.RCP/RSQRT	FP32 or integer reference	Activations, reciprocals, and inverse roots
Normalization	V.RMSNORM/L2NORM V.SOFTMAX	FP32 or integer reference	Norms and attention weights
Encoding	V.QUANT.A8/POW2 V.DECAY.ENCODE	Named quantization or gate profile	Activation codes, fraction metadata, gates

Profiles retain the repo's reference semantics or specify FP32 arithmetic

Block scaling will be included; its implementation remains undecided



Current status

- Each PE will support a 2×2 INT4 multiply or an INT8 multiply
- Q8.24 recurrence products will be computed serially
- The vector command contract covers SIMD arithmetic and composed operations
- Undecided: block scaling, compute sizing, buffering, and multiplier scheduling, final ISA, instruction format